



SOPORTE TEMÁTICO

Si tienes más de un tema por desarrollar, copia y pega la tabla por cada uno.

Nombre del tema 1:	Exploración de datos (EDA)
Desarrollo	del tema 1:



¡Hola coder, ahora vas a empezar a pensar como un analista de datos de verdad!

Hasta este punto has construido una base muy sólida. Entendiste qué son los datos, cómo se transforman en información y conocimiento, cómo fluyen dentro de una organización y, lo más importante, ya comenzaste a trabajar con ellos usando Python y Pandas.

Ya sabes cargar datos, transformarlos, filtrarlos y hacer cálculos. Es decir, ya tienes herramientas.

Pero ahora viene un cambio clave.

En el mundo real, un analista no empieza limpiando datos, ni haciendo gráficos, ni sacando conclusiones.

Empieza entendiendo los datos.

Y aunque suene simple, este es uno de los pasos más importantes de todo el proceso.

¿Qué es la exploración de datos (EDA)?

La exploración de datos, o EDA (Exploratory Data Analysis), es el proceso de analizar un dataset para entender qué contiene, cómo está estructurado y qué problemas tiene antes de empezar a trabajar con él.

En otras palabras:

EDA es el momento en el que conoces tus datos antes de tomar decisiones sobre ellos.

Aquí no estás transformando ni limpiando, estás observando, cuestionando y entendiendo.

¿Por qué es tan importante?

Porque los datos rara vez vienen perfectos.

En la práctica, un dataset puede tener:

- valores faltantes
- datos duplicados
- formatos incorrectos
- información inconsistente



- columnas que no tienen sentido

Si empiezas a analizar sin entender esto, puedes llegar a conclusiones incorrectas.

Por eso:

Un buen análisis comienza con una buena exploración.

¿Por qué es tan importante?

Porque los datos rara vez vienen perfectos.

En la práctica, un dataset puede tener:

- valores faltantes
- datos duplicados
- formatos incorrectos
- información inconsistente
- columnas que no tienen sentido

Si empiezas a analizar sin entender esto, puedes llegar a conclusiones incorrectas.

Por eso:

Un buen análisis comienza con una buena exploración.

Ejemplo: cargar y visualizar datos

```
1 import pandas as pd
2
3 # Cargar el set de datos a explorar
4 df = pd.read_csv("ventas.csv")
5
6 # Visualizar las primeras filas del dataset
7 print(df.head())
```

Esto te permite ver las primeras filas y tener una idea inicial de los datos.

Entender la estructura del dataset

Después de tener un primer vistazo de los datos, el siguiente paso natural no es transformarlos ni analizarlos directamente, sino entender cómo están organizados.

Este momento es clave porque aquí defines cómo vas a trabajar con el dataset en adelante. Si no entiendes la estructura, cualquier operación posterior puede estar mal planteada.

Cuando hablamos de estructura, nos referimos a responder preguntas como:

- ¿Cuántos registros tengo realmente?
- ¿Cuántas variables estoy manejando?
- ¿Cómo están representados esos datos internamente?
- ¿Cada columna tiene sentido dentro del contexto del negocio?

Este paso te permite construir un mapa mental del dataset. Ya no estás viendo datos sueltos, estás viendo un sistema de información.

Aquí es donde Pandas te da herramientas muy potentes para inspeccionar rápidamente esta estructura.



Ejemplo: estructura del dataset

```
1 import pandas as pd
2
3 # Cargar el set de datos a explorar
4 df = pd.read_csv("ventas.csv")
5
6 # Visualizar las filas y columnas (cantidad)
7 print(df.shape)
8
9 # Visualizar información detallada
10 print(df.info())
```

Tipos de datos: algo más importante de lo que parece

Una de las decisiones más importantes en analítica de datos no es el modelo ni la visualización, es algo mucho más básico:

trabajar con los tipos de datos correctos

Esto es crítico porque define qué operaciones puedes realizar.

Por ejemplo:

- sí un número está guardado como texto, no puedes sumarlo correctamente
- sí una fecha está como string, no puedes analizar tendencias en el tiempo

Este es uno de los errores más comunes en perfiles junior: asumir que los datos están bien tipados.

En la práctica, esto casi nunca ocurre.

Por eso, revisar los tipos de datos no es opcional, es obligatorio.



```
1 import pandas as pd
2
3 # Cargar el set de datos a explorar
4 df = pd.read_csv("ventas.csv")
5
6 # Visualizar el tipo de dato para cada columna del DF
7 print(df.dtypes)
```

Conclusiones del tema 1:

En esta lección entendiste que antes de limpiar, transformar o analizar, necesitas conocer profundamente tus datos.

EDA no es un paso técnico, es un paso estratégico.

Es el momento donde decides cómo vas a abordar el análisis y qué tan confiables son los datos que tienes.

Si omites este paso, puedes llegar a conclusiones completamente equivocadas, aunque tu código sea correcto.

Nombre del tema 2:	Limpieza y transformación de datos
Desarrollo del tema 2:	



www.rwi.io



301 732 5327



Cl. 16 # 55 - 129



¡Hola coder, ahora vas a preparar los datos para que realmente sirvan!

En la lección anterior diste un paso clave: aprendiste a explorar los datos. Ya sabes cómo ver su estructura, identificar tipos, detectar valores nulos y empezar a entender su comportamiento.

Pero ahora viene una de las etapas más importantes en todo el proceso de analítica.

Porque entender los datos no es suficiente.

Los datos deben estar correctos y listos antes de poder analizarlos.

Y aquí aparece una realidad que debes interiorizar desde ya:

los datos nunca vienen listos para ser analizados.

Siempre tienen problemas, inconsistencias o necesitan ajustes.

Por eso, en esta lección vas a aprender a:

- limpiar datos
- transformarlos
- prepararlos

para que realmente puedan generar valor.

Limpieza de datos: asegurar confiabilidad

Cuando hablas de limpieza, estás hablando de calidad.

Estás revisando si los datos:

- están completos
- son consistentes
- no tienen errores evidentes

Aquí tu objetivo no es analizar, es **corregir**.

Porque un dato incorrecto puede llevar a una conclusión incorrecta.

Transformación de datos: preparar para análisis

Cuando transformas, ya no estás corrigiendo, estás construyendo.



Estás adaptando los datos para responder preguntas.

Aquí haces cosas como:

- crear nuevas variables
- cambiar formatos
- agrupar información

Valores nulos: cuando la información no existe

Un valor nulo no es simplemente un vacío.

Es información que no está.

Y eso puede significar muchas cosas:

- el dato no se registró
- el dato no existe
- hubo un error en el sistema

La clave aquí no es solo detectarlos, sino entender su impacto.

Un valor nulo en una columna crítica puede invalidar un análisis completo.


```
1 import pandas as pd
2
3 # Creamos un dataset con errores reales (nulos, duplicados, etc.)
4 df = pd.DataFrame([
5     {"cliente": "Juan", "ciudad": "Medellín", "ventas": 100},
6     {"cliente": "Ana", "ciudad": "Bogotá", "ventas": None},
7     {"cliente": "Pedro", "ciudad": "Cali", "ventas": 150},
8     {"cliente": "Juan", "ciudad": "Medellín", "ventas": 100},
9     {"cliente": "Luisa", "ciudad": "Bogotá", "ventas": 300}
10 ])
11
12 # Detectar valores nulos en cada columna
13 # isnull() identifica valores faltantes
14 # sum() cuenta cuántos hay
15 print(df.isnull().sum())
```

Decisión analítica

Aquí debes decidir:

- ¿elimino el dato?
- ¿lo reemplazo?
- ¿lo ignoro?

No hay una respuesta universal. Todo depende del contexto.

Ejemplo: reemplazar por promedio

```

1 import pandas as pd
2
3 # Creamos un dataset con errores reales (nulos, duplicados, etc.)
4 df = pd.DataFrame([
5     {"cliente": "Juan", "ciudad": "Medellín", "ventas": 100},
6     {"cliente": "Ana", "ciudad": "Bogotá", "ventas": None},
7     {"cliente": "Pedro", "ciudad": "Cali", "ventas": 150},
8     {"cliente": "Juan", "ciudad": "Medellín", "ventas": 100},
9     {"cliente": "Luisa", "ciudad": "Bogotá", "ventas": 300}
10 ])
11
12 # Detectar valores nulos en cada columna
13 # isnull() identifica valores faltantes
14 # sum() cuenta cuántos hay
15 print(df.isnull().sum())
16
17 # Calculamos el promedio de ventas (ignorando nulos)
18 promedio = df["ventas"].mean()
19
20 # Reemplazamos los valores nulos con ese promedio
21 df["ventas"] = df["ventas"].fillna(promedio)
22
23 print(df)

```

Duplicados: cuando la realidad se repite (o parece hacerlo)

Un duplicado no siempre es evidente.

Puede parecer un dato válido, pero en realidad representa el mismo evento registrado dos veces.

Esto genera problemas como:

- inflar ventas
- duplicar clientes
- distorsionar métricas

```
1 import pandas as pd
2
3 # Creamos un dataset con errores reales (nulos, duplicados, etc.)
4 df = pd.DataFrame([
5     {"cliente": "Juan", "ciudad": "Medellín", "ventas": 100},
6     {"cliente": "Ana", "ciudad": "Bogotá", "ventas": None},
7     {"cliente": "Pedro", "ciudad": "Cali", "ventas": 150},
8     {"cliente": "Juan", "ciudad": "Medellín", "ventas": 100},
9     {"cliente": "Luisa", "ciudad": "Bogotá", "ventas": 300}
10 ])
11
12 # Contar registros duplicados
13 print("Duplicados:", df.duplicated().sum())
14
15 # Eliminar duplicados
16 # Esto garantiza que cada registro sea único
17 df = df.drop_duplicates()
18
19 print(df)
```

Filtrado: definir qué es relevante

En analítica de datos, una de las habilidades más importantes no es saber usar herramientas, sino saber **qué datos usar**.

Un error muy común en perfiles junior es intentar trabajar con todo el dataset al mismo tiempo. Esto no solo hace el análisis más complejo, sino que muchas veces lo vuelve confuso e irrelevante.

En la práctica, un analista no trabaja con todos los datos.

Trabaja con los datos que responden una pregunta específica.

Por eso, filtrar no es solo una operación técnica, es una decisión estratégica.



```
1 import pandas as pd
2
3 # Creamos un dataset con errores reales (nulos, duplicados, etc.)
4 df = pd.DataFrame([
5     {"cliente": "Juan", "ciudad": "Medellín", "ventas": 100},
6     {"cliente": "Ana", "ciudad": "Bogotá", "ventas": None},
7     {"cliente": "Pedro", "ciudad": "Cali", "ventas": 150},
8     {"cliente": "Juan", "ciudad": "Medellín", "ventas": 100},
9     {"cliente": "Luisa", "ciudad": "Bogotá", "ventas": 300}
10 ])
11
12 # Filtrar solo los registros de Bogotá
13 df_bogota = df[df["ciudad"] == "Bogotá"]
14
15 print(df_bogota)
```

Integración de datos: ver el panorama completo

En analítica de datos, uno de los errores más comunes es pensar que toda la información necesaria para responder una pregunta está contenida en un solo dataset.

En la realidad, esto casi nunca ocurre.

Los datos dentro de una organización están distribuidos en múltiples fuentes:

- una tabla puede tener ventas
- otra puede tener información de clientes
- otra puede tener datos geográficos
- otra puede tener productos

Cada una de estas fuentes contiene solo una parte de la realidad.

Por eso, si analizas un dataset de forma aislada, estás viendo una visión incompleta.

```

1 import pandas as pd
2
3 # Creamos un dataset con errores reales (nulos, duplicados, etc.)
4 df = pd.DataFrame([
5     {"cliente": "Juan", "ciudad": "Medellín", "ventas": 100},
6     {"cliente": "Ana", "ciudad": "Bogotá", "ventas": None},
7     {"cliente": "Pedro", "ciudad": "Cali", "ventas": 150},
8     {"cliente": "Juan", "ciudad": "Medellín", "ventas": 100},
9     {"cliente": "Luisa", "ciudad": "Bogotá", "ventas": 300}
10 ])
11
12 # Dataset adicional con información de contexto
13 df_ciudades = pd.DataFrame([
14     {"ciudad": "Medellín", "region": "Antioquia"},
15     {"ciudad": "Bogotá", "region": "Cundinamarca"},
16     {"ciudad": "Cali", "region": "Valle"}
17 ])
18
19 # merge() combina los datasets
20 # on="ciudad" indica la columna en común
21 df_final = df.merge(df_ciudades, on="ciudad")
22
23 print(df_final)

```

Conclusiones del tema 2:

En esta lección entendiste algo que marca una diferencia enorme entre alguien que usa herramientas y alguien que realmente hace analítica de datos.

La analítica no comienza cuando haces gráficos, ni cuando escribes consultas, ni cuando presentas resultados.

Comienza mucho antes.

Comienza cuando decides con qué datos vas a trabajar y qué tan confiables son.

A lo largo de esta lección viste que los datos, tal como llegan, no son una representación perfecta de la realidad. Son una aproximación, muchas veces incompleta, inconsistente o desordenada.



Por eso, el verdadero trabajo del analista no es simplemente ejecutar código, sino tomar decisiones sobre los datos:

- decidir qué datos son relevantes
- identificar qué información está incompleta o es incorrecta
- determinar cómo corregir esos problemas
- transformar los datos para que puedan responder preguntas reales

Aquí es donde se construye la calidad del análisis.

Porque puedes tener el mejor modelo, el mejor dashboard o la mejor visualización, pero si los datos no fueron preparados correctamente, el resultado será incorrecto.

Y lo más importante:

un análisis incorrecto no solo es un error técnico, es una mala decisión basada en información equivocada.

Nombre del tema 3:	SQL para análisis de datos
Desarrollo del tema 3:	



¡Hola coder, ahora vas a entrar al corazón de los datos en una organización!

Hasta este punto has trabajado con datos desde una perspectiva muy controlada.

Has usado Pandas, donde tú cargas los datos, los transformas y los analizas en memoria. Esto es excelente para aprender y para análisis exploratorios.

Pero en el mundo real, los datos no funcionan así.

Los datos:

- no están en archivos
- no los controlas completamente
- no puedes simplemente cargarlos todos

Los datos viven en sistemas.

Y esos sistemas están diseñados para:

- almacenar grandes volúmenes
- garantizar integridad
- permitir acceso controlado

Aquí es donde entra SQL.

Pero no el SQL básico.

¿qué es SQL en el contexto real?

Muchos piensan que SQL es solo un lenguaje para consultar datos. Pero en analítica, SQL es mucho más que eso.

SQL es:

- una forma de acceder a los datos
- una forma de transformarlos
- una forma de modelar resultados
- una forma de construir lógica de análisis

A medida que los problemas de análisis crecen, también lo hace la complejidad de las consultas.



Ya no estás respondiendo preguntas simples como:

- ¿cuánto vendí?

Ahora estás respondiendo preguntas como:

- ¿cuáles son las ciudades con mayor crecimiento en ventas en los últimos meses?
- ¿qué clientes tienen un comportamiento atípico?
- ¿cómo se distribuyen las ventas por región y segmento?

Responder este tipo de preguntas en una sola consulta plana se vuelve difícil, poco legible y propenso a errores.

Aquí es donde necesitas estructurar la lógica.

Organizar la lógica: el rol de las CTE

Cuando trabajas con múltiples pasos dentro de una consulta, necesitas una forma de organizar esa lógica.

Las CTE (Common Table Expressions) existen precisamente para eso.

No son simplemente una característica del lenguaje, son una herramienta para pensar mejor.

Te permiten dividir un problema grande en partes más pequeñas.

En lugar de escribir una consulta compleja y difícil de entender, puedes estructurarla en bloques:

- primero defines un conjunto de datos
- luego lo transformas
- luego aplicas otra lógica sobre ese resultado

Esto no solo mejora la legibilidad, también mejora tu capacidad de razonar sobre el análisis.


```

1  -- Definimos una CTE (subconsulta con nombre)
2  WITH ventas_por_ciudad AS (
3
4      -- Agrupamos ventas por ciudad
5      SELECT
6          ciudad,
7          SUM(ventas) AS total_ventas -- Sumamos ventas por cada ciudad
8      FROM ventas
9      GROUP BY ciudad
10
11 )
12
13 -- Usamos la CTE como si fuera una tabla
14 SELECT *
15 FROM ventas_por_ciudad
16 WHERE total_ventas > 200; -- Filtramos ciudades con ventas altas

```

Persistencia intermedia: tablas temporales

A medida que los análisis crecen, hay momentos donde no basta con definir lógica dentro de una consulta.

Necesitas guardar resultados intermedios.

Esto ocurre cuando:

- el cálculo es costoso
- necesitas reutilizar resultados
- quieres dividir el análisis en etapas

Aquí aparecen las tablas temporales.

Una tabla temporal no es simplemente una tabla que desaparece después. Es una forma de materializar un paso dentro de tu proceso analítico.

Es equivalente a cuando en Pandas guardas un DataFrame intermedio para seguir trabajando sobre él.

Este concepto es importante porque introduce algo clave:

el análisis no siempre ocurre en una sola consulta, puede ser un proceso compuesto por múltiples pasos.

```
1 -- Creamos una tabla temporal
2 CREATE TEMP TABLE temp_ventas_altas AS
3
4 -- Guardamos solo ventas relevantes
5 SELECT *
6 FROM ventas
7 WHERE ventas > 100;
8
9 -- Usamos la tabla temporal en otro análisis
10 SELECT
11     ciudad,
12     COUNT(*) AS cantidad_registros
13 FROM temp_ventas_altas
14 GROUP BY ciudad;
```

Encapsular lógica: el rol de las vistas

A medida que trabajas con datos, empiezas a notar que hay ciertas transformaciones que repites constantemente.

Por ejemplo:

- ventas por ciudad
- ventas por región
- clientes activos

Si cada vez que necesitas esa información escribes la misma consulta, estás generando redundancia.

Aquí es donde entran las vistas.

Una vista no es solo una consulta guardada. Es una forma de encapsular lógica de negocio.

Cuando defines una vista, estás diciendo:

“esto es una forma estándar de ver estos datos”

Esto permite:

- reutilizar lógica
- mantener consistencia



- simplificar consultas

```
1  -- Creamos una vista (lógica reutilizable)
2  CREATE VIEW ventas_por_ciudad AS
3
4  SELECT
5      ciudad,
6      SUM(ventas) AS total_ventas
7  FROM ventas
8  GROUP BY ciudad;
9
10 -- Usamos la vista como si fuera una tabla
11 SELECT *
12 FROM ventas_por_ciudad;
```

Optimización y rendimiento: materialized views

En analítica, no solo importa obtener resultados correctos.

También importa obtenerlos de forma eficiente.

Algunas consultas pueden ser costosas porque:

- procesan grandes volúmenes de datos
- hacen múltiples agregaciones
- combinan varias tablas

Si ejecutas esas consultas constantemente, el sistema puede volverse lento.

Aquí es donde entran las materialized views.

A diferencia de una vista normal, que se calcula cada vez que la consultas, una materialized view guarda el resultado físicamente.

Esto introduce un concepto importante:

intercambias actualización en tiempo real por rendimiento.

```
1  -- Creamos una vista materializada (guarda resultados físicos)
2  CREATE MATERIALIZED VIEW ventas_resumen AS
3
4  SELECT
5      ciudad,
6      SUM(ventas) AS total
7  FROM ventas
8  GROUP BY ciudad;
9
10 -- Consultamos como si fuera una tabla
11 SELECT *
12 FROM ventas_resumen;
13
14 -- Actualizamos los datos manualmente
15 REFRESH MATERIALIZED VIEW ventas_resumen;
```

Lógica dentro del sistema: funciones y procedimientos

A medida que avanzas, empiezas a mover parte de la lógica hacia la base de datos.

Esto es algo muy importante en entornos profesionales.

No todo el procesamiento ocurre en el cliente (Python, aplicaciones, etc.).

Parte de la lógica vive en el servidor.

Las funciones y procedimientos permiten:

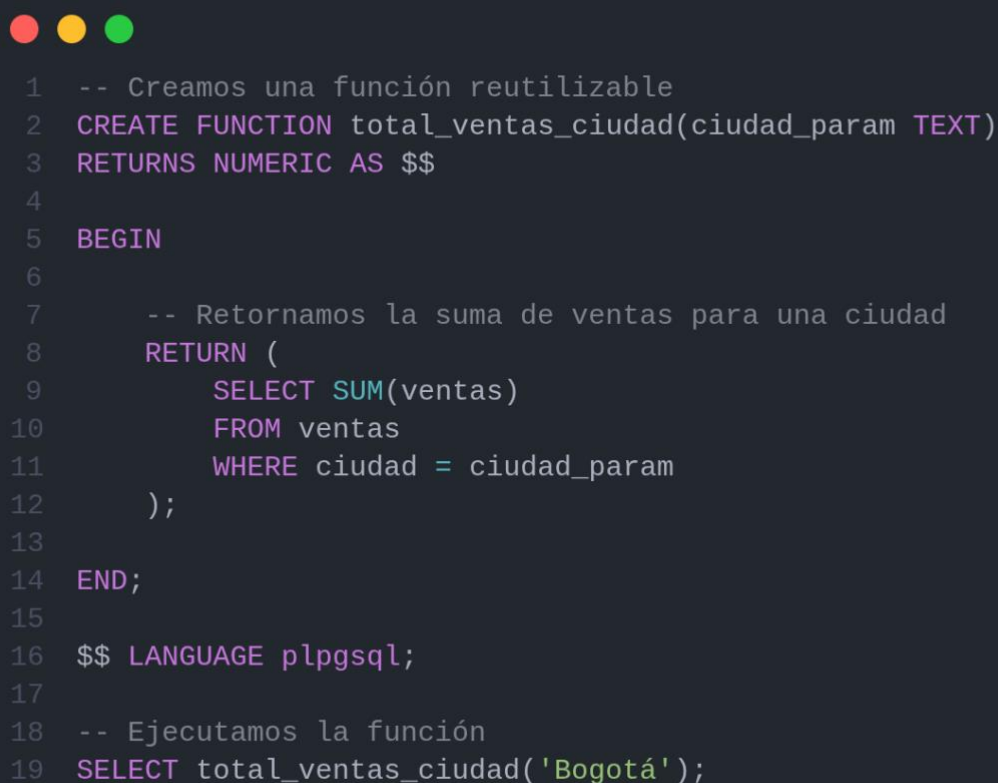
- encapsular cálculos
- reutilizar lógica
- estandarizar procesos

Esto convierte la base de datos en algo más que un almacenamiento.

La convierte en un sistema activo.

Ejemplo de procedimiento almacenado:

Ejemplo de Función



```

1  -- Creamos una función reutilizable
2  CREATE FUNCTION total_ventas_ciudad(ciudad_param TEXT)
3  RETURNS NUMERIC AS $$
4
5  BEGIN
6
7      -- Retornamos la suma de ventas para una ciudad
8      RETURN (
9          SELECT SUM(ventas)
10         FROM ventas
11         WHERE ciudad = ciudad_param
12      );
13
14  END;
15
16  $$ LANGUAGE plpgsql;
17
18  -- Ejecutamos la función
19  SELECT total_ventas_ciudad('Bogotá');

```

Conclusiones del tema 3:

A lo largo de esta lección diste un paso que marca una diferencia importante en tu formación como profesional de datos.

Hasta ahora, habías trabajado con los datos desde una perspectiva más controlada. Cargabas información, la explorabas, la limpiabas y la transformabas utilizando herramientas como Pandas. Esto te permitió entender cómo se comportan los datos y cómo prepararlos para análisis.

Sin embargo, en esta lección cambiaste completamente el contexto.

Dejaste de trabajar con datos que tú controlas y empezaste a trabajar con datos que viven dentro de un sistema.

Y eso cambia todo.



Porque en un entorno real, los datos:

- no están organizados para ti
- no están listos para ser usados directamente
- no pueden ser movidos libremente
- y muchas veces no pueden ser cargados completamente en memoria

Por eso, el análisis no ocurre únicamente fuera de la base de datos.